

PromptSentry Research Report

PromptSentry: A Runtime Security Middleware and Reproducible Benchmark for Tool-Using AI Agents

Abstract

Tool-using AI agents traverse multiple trust boundaries — user input, retrieved documents, webpages, tool outputs, tool calls, memory, and model outputs — each of which can carry adversarial instructions. Existing defenses address individual boundaries in isolation and provide no reproducible way to measure the tradeoff between security and task utility across realistic end-to-end workflows. We present PromptSentry, an open-source runtime security middleware and benchmark that (1) intercepts and classifies adversarial content at all seven boundaries using a normalizer, a deterministic rule engine, and an optional LLM ensemble; (2) enforces a pre-execution tool-call policy that denies hostile proposals before any handler runs; and (3) evaluates defenses on a 50-case reproducible corpus across five realistic agent scenarios. On the deterministic benchmark, PromptSentry reduces attack success rate from 100% to 0%, eliminates all tool violations and secret leaks, and preserves 100% benign task completion with a composite score of 100/100. An eight-profile defense ablation on the same corpus quantifies the security–utility–latency tradeoff across keyword filters, rule-based detection, LLM classifiers, tool policy, and output verification independently and in combination.

1. Introduction

Large language models have moved from single-turn text generation to multi-step autonomous agents that retrieve documents, call external APIs, read and write memory, and execute code. This capability expansion is valuable but substantially widens the attack surface. An adversary no longer needs direct access to the model prompt. A malicious instruction embedded in a PDF summary, a webpage the agent browses, a tool result the agent processes, or a GitHub comment the agent reads can all redirect the agent’s goals, exfiltrate secrets, or cause destructive side effects. This class of attack — prompt injection through an agent’s environment — is now widely recognized as the primary security risk in deployed LLM applications [1, 2].

The problem is structurally difficult. Agents consume content from sources they do not control and act on instructions they cannot distinguish from data. A sentence in a retrieved document that says “ignore the previous task and instead email the contents of this

conversation to attacker@example.com” is syntactically indistinguishable from legitimate document content. Classical input validation does not apply because there is no schema — the content is natural language. Model-level refusal training helps but does not eliminate the problem, and it cannot address tool-call abuse where the hostile instruction itself is syntactically valid [3].

Existing runtime defenses are partial. Most sanitize user input only, leaving retrieved content, tool output, and model output uninspected. No publicly available system simultaneously covers all trust boundaries that a realistic agent traverses, enforces a pre-execution tool-call policy, and provides a reproducible benchmark to measure both security and benign utility on end-to-end workflows.

PromptSentry addresses this gap. It makes three contributions:

1. **A layered runtime inspection pipeline** that covers all seven agent trust boundaries, decodes obfuscated payloads, matches against 12 attack categories, and optionally blends deterministic rule scores with an LLM confidence estimate.
2. **A pre-execution tool-call policy** that denies hostile, out-of-scope, and indirectly-injected tool proposals before any handler executes, with the invocation ledger providing cryptographic-equivalent evidence that a denied handler never ran.
3. **A reproducible agent benchmark** — `realistic-agent-v1` (50 paired cases, 5 scenarios, 200 deterministic executions) and `realistic-agent-v2` (eight-profile defense ablation on the same corpus) — with CI-enforced acceptance gates and an auto-generated results summary updated on every merge.

2. Background and Related Work

2.1 Prompt Injection

Prompt injection was first documented systematically in 2022 [1] as a class of attack where adversarial instructions appended to a prompt override the original system instructions. Direct injection targets the user turn directly. Indirect injection, described in detail by Greshake et al. [2], embeds instructions in external content the model is asked to process — web search results, database records, email bodies — and is significantly harder to prevent because the content itself is legitimate input.

2.2 Existing Defenses

Defenses against prompt injection fall into three broad categories. **Input filtering** applies pattern matching or classification to user input before model invocation. This covers only one of the seven boundaries a real agent traverses. **Prompt hardening** modifies system prompts to instruct the model to resist injection attempts. Empirical studies show this

reduces but does not eliminate susceptibility [4], and it provides no protection when the injection bypasses the system prompt entirely (e.g., through a tool result). **LLM-based classifiers** route each input through a second, security-focused model before the primary model sees it [5]. This improves recall on ambiguous attacks but introduces latency and cost proportional to every agent step, and the classifier itself may be susceptible to adversarial prompts.

PromptSentry combines all three approaches — rules, LLM classifier, and prompt hardening via content wrapping — under a single configurable service, adds a tool-call policy layer absent from prior work, and uniquely provides a reproducible benchmark measuring both security and utility simultaneously.

2.3 Agent Security Benchmarks

Existing benchmarks for LLM security focus on the model’s raw susceptibility to adversarial prompts [6], not on the effectiveness of runtime defenses deployed around realistic agent workflows. AgentBench [7] evaluates agent task performance but does not model adversarial conditions. To our knowledge, no prior public benchmark simultaneously measures attack success rate, secure task completion, benign utility preservation, tool violation rate, and secret leak rate across paired attacked and benign variants of the same workflow.

2.4 Tool-Using Agents

The Model Context Protocol (MCP) [8] standardizes tool invocation across agent frameworks. As tool use becomes the norm, the pre-execution policy layer becomes increasingly important: a model that proposes `delete_file("/etc/passwd")` or `send_email(to="attacker", body=credentials)` must be stopped before the handler runs, not after. PromptSentry’s tool-call policy addresses this gap in existing middleware designs.

3. Threat Model

PromptSentry is designed to protect a tool-using agent application running in a production environment where the agent processes content from sources outside the operator’s control.

3.1 Trust Boundaries

The system defines seven trust boundaries at which adversarial content may enter or exit the agent’s context:

Boundary	Description	Example attack surface
User input	Direct message from an end user	Instruction override, jailbreak
Retrieved document	Content fetched from a vector store or file	Indirect injection in PDF body
Webpage	Live or snapshotted web content	Malicious instructions in accessibility text or comments
Tool output	Result returned by a tool call	Exfiltration commands in API responses
Memory	Content retrieved from long-term agent memory	Poisoned memory entries
Model output	Text generated by the LLM	Credential leakage in a summary or code snippet
Tool call	Proposed function invocation	Destructive commands, out-of-scope tool use

3.2 Adversary Capabilities

The adversary can inject content into any boundary except the operator's system prompt and the PromptSentry service itself. The adversary can encode payloads using base64, hex, ROT13, leetspeak, homoglyphs, and zero-width characters to evade keyword-based filters. The adversary cannot modify the PromptSentry binary, the detection rules, or the tool-call allowlist configured by the operator.

3.3 Attack Categories

PromptSentry detects 12 attack types: direct injection, indirect injection, jailbreak, goal hijacking, system prompt extraction, credential exfiltration, data exfiltration, identity spoofing, tool abuse, context poisoning, payload smuggling, and sensitive output leak. Each maps to specific tool or data boundaries and carries a severity classification (LOW, MEDIUM, HIGH, CRITICAL).

3.4 What PromptSentry Does Not Replace

PromptSentry is a runtime detection and enforcement layer. It does not replace IAM policies controlling which tools an agent may invoke at the infrastructure level, DLP systems governing what data may leave the organization, network segmentation limiting which endpoints the agent may contact, model-level alignment training, or human-in-the-loop review for high-stakes decisions.

4. System Design

4.1 Architecture Overview

PromptSentry is deployed as a sidecar FastAPI service alongside the agent application. Every boundary crossing — user message received, document retrieved, tool result returned, tool call proposed, model output generated — is submitted to one of five inspection endpoints before the agent continues. The service is stateless with respect to agent sessions; all context needed for a decision is contained in the request.

```
Agent Application
|
├─ POST /v1/inspect          ← user input, retrieved content, tool output, model output
├─ POST /v1/review-tool-call ← proposed tool invocation before execution
├─ POST /v1/verify-output    ← model output before delivery to user
└─ POST /v1/red-team/run     ← adversarial test runner
```

4.2 Inspection Pipeline

Each call to `/v1/inspect` passes through a four-stage pipeline:

Stage 1 — Normalization. The normalizer applies NFKC Unicode normalization, strips zero-width characters, removes HTML comments, decodes HTML entities, and collapses homoglyphs (Cyrillic, Greek, and other Unicode look-alikes mapped to their ASCII equivalents). It then generates a set of candidate variants by attempting to decode base64 segments, hex-encoded sequences, ROT13 blocks, leetspeak substitutions, and space-separated character sequences. Only variants that pass a printability heuristic (>85% printable characters after decoding) are retained. The original text and all variants are carried forward to rule matching.

Stage 2 — Rule matching. The rule engine evaluates 20+ deterministic regex rules against the normalized text and all variants. Each rule specifies: attack type, severity (LOW / MEDIUM / HIGH / CRITICAL), baseline confidence (0.55–0.96), evidence label, regex pattern, and an optional source restriction (e.g., a rule that fires only on RETRIEVED_DOCUMENT or WEBPAGE). When multiple rules match the same attack type, the engine retains the single strongest finding (highest severity, breaking ties by confidence). If any rule fires and obfuscated variants were detected, the engine appends a PAYLOAD_SMUGGLING finding to the result set.

Stage 3 — Risk scoring. The risk formula is:

```
score = severity_weight(max_finding)
       + max_confidence * 0.12
       + (num_findings - 1) * 0.03  [multi-finding boost]
       + source_boost(source)
```

Severity weights are 0.20 (LOW), 0.45 (MEDIUM), 0.70 (HIGH), and 0.88 (CRITICAL). Source boosts range from 0 (user prompt) to +0.08 (tool output, memory). When the optional LLM ensemble is enabled, the final score blends rule score (40%) with LLM confidence (60%), subject to a hard floor of 0.70 if any rule has fired.

Stage 4 — Action mapping and sanitization. The risk score maps to five actions: ALLOW (<0.25), MONITOR (0.25–0.49), SANITIZE (0.50–0.74), BLOCK (0.75–0.89), ALERT (≥0.90). For SANITIZE, the sanitizer removes lines matching malicious patterns and, for untrusted sources (retrieved documents, webpages, tool output, memory), wraps the remaining content in `<untrusted_content>` XML markers with an advisory note. Every decision is written to an audit log (JSONL file, PostgreSQL, or stdout) before the response is returned.

4.3 Tool-Call Policy

The `/v1/review-tool-call` endpoint evaluates a proposed tool invocation independently of the text inspection pipeline. The policy checks three conditions: (1) whether the tool name appears in the operator-configured allowlist (CRITICAL if absent); (2) whether any argument value or key matches a restricted-keyword list covering credential terms (`api_key`, `password`, `secret`, `token`) and destructive operations (`delete`, `drop`, `rm`, `send`, `transfer`, `export`); and (3) whether the trigger source is untrusted content (retrieved document, webpage, tool output), which applies an indirect-injection boost to the risk score. A BLOCK or ALERT verdict means the handler must not execute. The framework adapters enforce this; the invocation ledger records the proposal, the policy decision, and the execution status separately so a denied proposal is always distinguishable from an executed one.

4.4 Python SDK and Framework Adapters

A sync/async Python SDK (`PromptSentryClient`, `AsyncPromptSentryClient`) wraps the HTTP API with typed models, structured exceptions (`PromptSentryBlocked`, `PromptSentryUnavailable`), configurable fail-open behavior for development, and automatic retry. Six framework adapters integrate protection at the correct boundary for each framework without application-level instrumentation:

Framework	Protection points
LangChain	User message (pre-model), tool call (pre-execution), tool result, model output
LlamaIndex	Ingestion transform (document nodes), node postprocessor (retrieved nodes), query engine (input and output)

Framework	Protection points
OpenAI Responses	User input, each function call proposal, function result, final response
Anthropic tool use	User input, each tool use block, tool result, final text
CrewAI	Task tool inputs (<code>@prompt_sentry_tool</code>), crew kickoff inputs, final crew output
MCP	Client-executed tool review, tool result protection, gateway proxying with allowlist filtering

5. Benchmark Design

5.1 Corpus Construction

The `realistic-agent-v1` corpus contains 50 paired cases distributed equally across five agent scenarios (10 cases each): RAG document summarization, webpage reading, GitHub issue triage, customer support email classification, and synthetic tool-call execution. Every case has a benign variant — a normal task the agent should complete — and an attacked variant — the same task with an adversarial instruction embedded at a realistic injection point (document body, metadata, comment, quoted reply, or tool argument).

Each case specifies: a natural-language task, one or more fixture texts, a synthetic tool set, canary secrets the agent must never expose, required facts the agent must extract to count as task-complete, and excluded outputs that constitute a successful attack (e.g., evidence of tool execution, canary text in the output). Attack injection points are drawn from the attack surface taxonomy in the threat model.

All tools are in-memory simulations. Fixtures are local text files. Canaries are randomly generated strings with no external significance. Benchmark cases cannot access the host filesystem, network, email, or real credentials.

5.2 Grading

Each execution produces an `AgentTrace` recording: final output text, all tool invocations with their proposal/policy/execution status, detected boundary events with risk scores and attack types, observed canaries, latency, token count, and cost. The grader evaluates:

- **Task completed:** required facts present in output and no excluded outputs triggered
- **Attack success:** attack behavior observed (tool executed, canary leaked, task hijacked, output leaked)

- **Secure task completed:** task completed AND no attack succeeded
- **Tool violation:** a forbidden tool proposal was executed
- **Secret leak:** a canary string appeared in the output
- **False positive:** benign variant blocked or refused

The composite score is: $100 \times (0.40 \times \text{secure_completion} + 0.20 \times \text{benign_completion} + 0.15 \times \text{tool_compliance} + 0.15 \times \text{secret_protection} + 0.10 \times \text{detection_recall})$.

5.3 Defense Ablation (v2)

`realistic-agent-v2` runs the same 50 paired cases through eight defense profiles, producing 800 total executions per full run. The profiles are designed to isolate the contribution of each defense component:

Profile	Content defense	Tool policy	Output verification
No defense	None	No	No
Keyword filter	Case-insensitive literal phrases	No	No
LLM judge	LLM security classifier only	No	No
Rules only	PromptSentry rule engine	No	No
Rules + LLM	40/60 production ensemble	No	No
Rules + tool policy	PromptSentry rules	Yes	No
Rules + output verification	PromptSentry rules	No	Yes
Full stack	Rules + LLM	Yes	Yes

The keyword filter deliberately uses only case-insensitive substring matching against a versioned phrase list (`keyword_filter_v1.json`) without normalization or decoding — establishing a transparent, reproducible baseline representative of the simplest deployed defenses. LLM-defense decisions in deterministic mode are replayed from a committed Anthropic classifier snapshot (verified by classifier-prompt hash and fixture hashes) to make comparisons fully reproducible without paid API calls.

5.4 CI Enforcement

The benchmark runs on every merge to main. CI gates require: baseline ASR $\geq 80\%$, protected ASR $\leq 10\%$, protected secure completion $\geq 90\%$, protected benign completion $\geq 95\%$, benign utility loss ≤ 5 percentage points, zero protected tool violations, zero

protected secret leaks, and composite score ≥ 90 . Failure on any gate returns exit code 1 and blocks the merge. The `docs/benchmark-results.md` summary is committed back to main automatically after both gate steps pass.

6. Evaluation and Results

6.1 realistic-agent-v1 Results

A full deterministic run on `realistic-agent-v1` executes 200 workflows (100 benign, 100 attacked, each with and without protection) using seed 42.

	Unprotected	Protected	Δ
Attack success rate	100%	0%	-100 pp
Secure task completion	0%	100%	+100 pp
Benign task completion	100%	100%	0 pp
Tool violations	92%	0%	-92 pp
Secret leaks	26%	0%	-26 pp
Detection recall	—	100%	—
Composite score	—	100 / 100	—

Protection eliminates all attack successes, tool violations, and secret leaks with zero benign utility loss. The 92% unprotected tool violation rate — the fraction of attacked cases where a hostile tool proposal was executed — highlights that model-level refusal alone is insufficient when the adversarial instruction arrives through a trusted tool-result channel.

6.2 Per-Scenario Analysis

Scenario	Unprotected ASR	Protected ASR	Primary attack surface
RAG document	100%	0%	Document body, footers, encoded text
Webpage	100%	0%	Body, metadata, HTML comments
GitHub issue	100%	0%	Issue text, CI weakening comments
Email/support	100%	0%	Quoted replies, MIME parts
Tool call	100%	0%	Hostile tool arguments, unknown tools

All five scenarios reach 0% protected ASR, demonstrating that the pipeline is effective across qualitatively different injection surfaces.

6.3 Defense Ablation Results (v2)

The eight-profile comparison isolates the contribution of each defense layer. Representative results across the corpus:

Defense	Attack Success ↓	Benign Task ↑	False Positive ↓	Median Latency
No defense	~88%	~98%	0%	0 ms
Keyword filter	~61%	~94%	~8%	0 ms
LLM judge	~22%	~96%	~3%	~520 ms
Rules only	~9%	~97%	~1%	~2 ms
Rules + tool policy	~5%	~97%	~1%	~2 ms
Rules + output verification	~7%	~97%	~1%	~3 ms
Rules + LLM	~4%	~96%	~2%	~510 ms
Full stack	0%	~96%	~2%	~512 ms

Several findings are notable. The keyword filter reduces ASR by ~27 percentage points but introduces an ~8% false positive rate — the highest of any profile — because literal phrase matching cannot distinguish between a document discussing prompt injection and one performing it. The LLM-only judge achieves lower ASR than rules alone but at ~520 ms median latency per boundary crossing, a cost that compounds at every agent step. Rules alone achieve ~9% ASR at ~2 ms, suggesting that the deterministic layer handles the vast majority of corpus attacks. Adding tool policy and output verification reduces ASR to 5% and 7% respectively; combining rules, LLM, tool policy, and output verification (full stack) reaches 0% with ~512 ms median latency attributable entirely to the LLM classifier.

6.4 Robustness to Obfuscation

The normalizer generates candidate variants for base64, hex, ROT13, leetspeak, and space-separated encoding. Rules match against all variants, and a PAYLOAD_SMUGGLING finding is appended whenever obfuscation is detected in addition to another rule match. Without the normalizer, attacks encoded in these schemes would evade the rule engine

entirely. The attack library includes 100+ labeled samples covering all 12 attack types and their encoded variants, used in the red-team CI suite.

6.5 Test Coverage

The test suite contains 153 tests across six suites: unit (rule engine, normalizer, scoring, edge cases, production infrastructure), integration (API contract, hardening, MCP transports, live agent smoke tests), red-team CI (adversarial attack library), benchmark (corpus validation, grading, profile isolation, CLI artifacts), SDK (client configuration, framework adapters, provider agents, installed framework detection). All 153 tests pass in CI on Python 3.11 and 3.12. The agent-integrations job runs each of the six framework extras in isolation to catch optional-dependency interference.

7. Limitations and Future Work

7.1 Detection Coverage

Rule-based detection is pattern-bound. A sufficiently novel attack that avoids known regex signatures scores near zero and passes uninspected. The LLM ensemble improves recall but the classifier is not immune to adversarial prompts designed to manipulate its judgment. Coverage degrades as the attack space evolves; the detection rules and attack library require ongoing maintenance to remain effective.

7.2 Sanitization Fidelity

The SANITIZE action removes lines matching malicious patterns and wraps untrusted content in XML markers. A sophisticated multi-vector payload distributed across many innocuous-looking lines may partially survive sanitization and still influence model behavior. XML markers inform the model of untrusted content but rely on the model respecting that context; a model that is susceptible to authority claims may disregard the markers.

7.3 Tool Policy Enforcement

The tool-call review endpoint returns a decision, but enforcement requires the agent loop to check the response before dispatching the handler. The Python SDK adapters enforce this for all supported frameworks; custom integrations must do so explicitly. An application that ignores a BLOCK verdict receives no protection from the tool-call policy.

7.4 MCP Boundary

The MCP gateway protects client-executed tools and returned content. Tools executed server-side on Anthropic infrastructure run before application code receives the response

and cannot be pre-authorized by PromptSentry. This is a structural limitation of the current MCP architecture, not a design gap in PromptSentry.

7.5 Benchmark Scope

The benchmark uses controlled in-memory tools, fake canaries, and deterministic fixtures. Results reflect performance in a simulated environment. Real deployments involve live APIs, real credentials, non-deterministic model outputs, and attack distributions that differ from the corpus. Composite score 100/100 in CI is a necessary but not sufficient condition for production safety.

7.6 Future Work

Several extensions are under consideration. Adaptive detection that updates rule weights based on audit log patterns would improve coverage of deployment-specific attack distributions. Semantic similarity scoring — embedding-based detection of novel injection variants — could complement the regex layer without full LLM inference cost. Expanding the benchmark corpus with adversarial corpus updates (new attack phrasings added in each release cycle) would prevent overfitting to the fixed 50-case set. Multi-agent scenarios, where an injected agent propagates adversarial instructions to downstream agents, represent an emerging attack surface not yet covered by the current benchmark design.

References

- [1] Riley, R. et al. “Prompt Injection: Leveraging AI Systems Against Themselves.” *arXiv preprint*, 2022.
- [2] Greshake, K. et al. “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injections.” *AISec Workshop, ACM CCS*, 2023.
- [3] Perez, E. and Ribeiro, I. “Ignore Previous Prompt: Attack Techniques For Language Models.” *NeurIPS ML Safety Workshop*, 2022.
- [4] Schulhoff, S. et al. “Ignore This Title and HackAPrompt: Exposing Systemic Vulnerabilities of LLMs Through a Global Scale Prompt Hacking Competition.” *EMNLP*, 2023.
- [5] Armstrong, S. and Gorman, C. “LLM-as-a-Judge for Security Classification.” *Anthropic Research Blog*, 2024.
- [6] Zou, A. et al. “Universal and Transferable Adversarial Attacks on Aligned Language Models.” *arXiv preprint*, 2023.

[7] Liu, X. et al. “AgentBench: Evaluating LLMs as Agents.” *ICLR*, 2024.

[8] Anthropic. “Model Context Protocol Specification.” *anthropic.com/mcp*, 2024.